



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

PROGRAMA ACADÉMICO DE CIENCIAS DE LA COMPUTACIÓN

CURSO: 1ACC0065 – PROGRAMACIÓN CONCURRENTES

NRC: 8646

PC1

INTEGRANTES:

Manchay Paredes, Lucero Salome (U202216120)

Mayhua Hinostroza, José Antonio (U202218044)

Peña Cardenas, Jhamil Brijan (U2017114492)

202620

ÍNDICE

1.- Presentar un informe entregable PC1 – Fecha de entrega semana 3:

a. Carátula

b. índice

c. Resumen del trabajo

d. Objetivos del trabajo

e. Explicación de cada tema investigado (3 temas por grupo)

Por cada paper deberá presentar una tabla con la siguiente información:

- Título del paper

- Autores

- Año de publicación

- Motivación

- Problema que resuelve

- Propuesta de solución

- Algoritmos usados, pseudocódigo

- Descripción de servicios, framework, bibliotecas usadas, código, lenguaje de programación, patrones.

- Opinión crítica de su parte (estudiante) de los beneficios o perjuicios del uso de la programación concurrente.

- Link de acceso al paper.

f. Análisis del caso de uso, sustentar su elección y explicación de la selección del dataset

g. Explicación del procedimiento de limpieza del dataset.

h. Trabajo en github, historial de commits de cada integrante del grupo.

i. Cada integrante del grupo sube el informe al aula virtual.

j. El coordinador del grupo adjunta en la entrega el reporte de participación del grupo.

1. Resumen del trabajo

El presente trabajo aborda la optimización de costos y recursos hospitalarios mediante el uso de modelos de machine learning implementados con mecanismos de concurrencia en el lenguaje Go. El caso de uso se enmarca en el Objetivo de Desarrollo Sostenible 3 (Salud y bienestar) y busca analizar los costos médicos y los días de estancia hospitalaria para apoyar la asignación eficiente de camas y presupuestos en centros de salud.

Se plantea el uso del modelo de Regresión Lineal, asignado y elegido libremente por el grupo, para predecir el costo total de atención (Total Costs) en función de los días de estancia (Length of Stay) y variables categóricas asociadas al diagnóstico y tipo de pago. El modelo será implementado de forma secuencial y concurrente (goroutines, sync.Mutex, sync.WaitGroup, patrones Worker Pool / Pipeline) sobre un dataset de más de un millón de registros, con el fin de medir el speedup obtenido y evaluar la escalabilidad de la solución.

Este entregable (PC1) documenta la investigación bibliográfica inicial, la sustentación del caso de uso, la selección del dataset y el procedimiento de limpieza de datos que servirá de base para los entregables posteriores (PC2 y TP).

2. Objetivos del trabajo

2.1 Objetivo general

Diseñar e implementar un algoritmo concurrente en Go que aplique el modelo de Regresión Lineal sobre un dataset hospitalario de más de un millón de registros, optimizando el tiempo de ejecución y evaluando su escalabilidad frente a una implementación secuencial equivalente.

2.2 Objetivos específicos

- Investigar el estado del arte sobre el uso de concurrencia en algoritmos de machine learning, identificando beneficios y limitaciones reportados en la literatura académica reciente.
- Seleccionar y depurar un dataset público de salud con más de un millón de registros, adecuado para el entrenamiento del modelo propuesto.
- Sustentar la elección del caso de uso y su vínculo con el ODS 3 (Salud y bienestar).
- Modelar la lógica de sincronización en Promela y verificarla formalmente en Spin (entregables posteriores).
- Implementar y comparar versiones secuenciales y concurrentes del modelo, midiendo el speedup y el uso de recursos de cómputo (entregables posteriores).

3. Explicación de los temas investigados

El grupo investiga dos ejes temáticos relacionados con el caso de uso: (1) programación concurrente en Go aplicada a machine learning, y (2) el modelo de Regresión Lineal y sus

variantes de paralelización. A continuación cada integrante documenta 3 papers académicos (últimos 5 años) por eje, siguiendo el formato exigido por la rúbrica.

3.1 Concurrencia en Go aplicada a Machine Learning

Go ofrece concurrencia nativa mediante goroutines (hilos ligeros gestionados por el runtime) y canales, junto con primitivas de sincronización como `sync.Mutex` (exclusión mutua) y `sync.WaitGroup` (espera de finalización de tareas). Estos mecanismos permiten implementar de forma concurrente el entrenamiento o la inferencia de modelos de ML sin depender de librerías externas, distribuyendo el dataset entre varios workers y agregando resultados parciales, típicamente mediante patrones Worker Pool o Pipeline.

3.2 Regresión Lineal concurrente

La Regresión Lineal, al basarse en operaciones matriciales y sumatorias independientes por bloque de datos, es un candidato natural para implementación concurrente mediante particionamiento de datos (data partitioning): cada goroutine calcula sumas parciales de forma concurrente e independiente (por ejemplo, para el método de mínimos cuadrados o el descenso de gradiente por lotes), que luego se combinan de forma sincronizada usando primitivas como `sync.Mutex` y `sync.WaitGroup`.

La concurrencia permite que múltiples goroutines trabajen sobre particiones del dataset de forma intercalada, mejorando la escalabilidad y el tiempo de ejecución en comparación con una implementación secuencial pura. En hardware multi-núcleo, esta concurrencia se traduce en paralelismo real, logrando un speedup medible.

3.3 Cadena de búsqueda utilizada

Siguiendo la indicación del docente, la búsqueda de los 3 papers académicos (Scopus, IEEE Xplore, ACM Digital Library) se realizó utilizando de forma obligatoria la siguiente cadena de búsqueda (string), exigiendo la presencia del término "Machine Learning" en combinación con "concurrent programming":

"Machine learning" AND "concurrent programming"

Cada integrante aplicó este mismo string como base en las tres fuentes indexadas indicadas, ajustando únicamente filtros de fecha (últimos 5 años: 2021-2026) y, de ser necesario, agregando términos adicionales propios de su subtema (por ejemplo: linear regression, Go, goroutines, gradient descent, `sync.Mutex`) sin remover los componentes obligatorios del string.

Ejemplo de refinamiento por integrante:

"Machine learning" AND "linear regression" AND "concurrent programming"

Integrante 1: José Antonio Mayhua Hinostroza

- **Paper:** Distributed SGD in overparametrized linear regression
- **Cadera de Búsqueda:** "machine learning" AND "linear regression" AND "distributed learning" AND "stochastic gradient descent"

Integrante 2: Lucero Salome Manchay Paredes:

- **Paper:** Recursive SQL and GPU-support for in-database machine learning
- **Cadera de Búsqueda:** "machine learning" AND "linear regression" AND ("parallel" OR "GPU") AND "gradient descent"

Integrante 3:

- **Paper:**
- **Cadera de Búsqueda:**

3.4 Tabla de papers por integrante

Cada integrante debe completar una fila por paper (1 paper por integrante, 3 en total), usando fuentes indexadas (Scopus, IEEE Xplore, ACM Digital Library) de los últimos 5 años.

Paper 1 (Integrante 1 - José Antonio Mayhua Hinostroza)

Campo	Contenido
Título del paper	Distributed SGD in overparametrized linear regression
Autores	Mike Nguyen, Charly Kirst y Nicole Mücke
Año de publicación	2024
Motivación	Los autores parten del problema de que el costo temporal y espacial de los métodos de aprendizaje, como SGD, puede volverse prohibitivo cuando aumenta el tamaño de los datos. Para abordar problemas de Machine Learning a gran escala, analizan una estrategia de aprendizaje distribuido basada en dividir los datos entre varios nodos computacionales y ejecutar localmente Stochastic Gradient Descent.
Problema que resuelve	El trabajo estudia cómo aplicar Stochastic Gradient Descent distribuido (DSGD) a Regresión Lineal, particularmente en escenarios sobreparametrizados, y analiza bajo qué condiciones el aumento del número de nodos sigue siendo beneficioso sin afectar el error de generalización. El problema es relevante porque distribuir el entrenamiento reduce la carga que tendría que asumir una sola máquina, pero dividir demasiado los datos puede perjudicar el comportamiento estadístico del modelo.

Propuesta de solución	El dataset de entrenamiento se divide de forma uniforme en M subconjuntos disjuntos, asignados a diferentes nodos o dispositivos. Cada nodo ejecuta de manera independiente SGD con paso constante sobre su subconjunto local. Al terminar el entrenamiento local, las estimaciones obtenidas por los diferentes nodos se envían a un servidor central, donde se promedian para construir el estimador global. Los autores analizan de forma teórica el exceso de riesgo y comparan DSGD con Distributed Ridge Regression (DRR).
Algoritmos usados / pseudocódigo	Regresión Lineal, Stochastic Gradient Descent (SGD), Distributed SGD (DSGD), promedio de iteraciones locales y agregación de modelos. Formalmente, cada nodo actualiza sus parámetros mediante SGD usando únicamente su partición de datos. Finalmente, los parámetros locales se promedian para generar el modelo global.
Servicios, frameworks, bibliotecas, lenguaje, patrones	El artículo es principalmente teórico y matemático, y no especifica un lenguaje de programación, framework de ML o biblioteca concreta para implementar el algoritmo. Sí define una arquitectura de aprendizaje distribuido de tipo divide-and-conquer, con cómputo local y agregación central: dataset dividido entre múltiples nodos, entrenamiento local mediante SGD y posterior promedio central de modelos. El artículo incluye experimentos numéricos con datos simulados y reales, pero no menciona en el texto revisado una tecnología específica como Python, TensorFlow o PyTorch.
Opinión crítica del estudiante	Considero que el aprendizaje distribuido propuesto presenta una ventaja importante para problemas con grandes volúmenes de datos, ya que permite dividir la carga computacional entre varios nodos y ejecutar el entrenamiento local de forma independiente. Esto puede mejorar la escalabilidad y reducir la carga de una única unidad de procesamiento. Sin embargo, el artículo muestra que aumentar sin control el número de nodos no garantiza una mejora: cuando el número de particiones crece demasiado respecto al tamaño de la muestra, el error del modelo puede aumentar. Por ello, la concurrencia debe diseñarse buscando un equilibrio entre paralelismo, cantidad de datos asignados a cada worker y calidad del modelo. Para nuestro proyecto, esta conclusión sugiere que aumentar el número de goroutines debe evaluarse de forma experimental en lugar de asumir que más workers siempre producirán un mayor speedup.
Link de acceso al paper	DOI: https://www.worldscientific.com/doi/10.1142/S021953052350032X

	Acceso al paper: https://openreview.net/forum?id=sfrcfYMONZ
--	--

Paper 2 (Integrante 2 - Lucero Salome Manchay Paredes)

Campo	Contenido
Título del paper	Recursive SQL and GPU-support for in-database machine learning
Autores	Maximilian E. Schüle, Harald Lang, Maximilian Springer, Alfons Kemper, Thomas Neumann y Stephan Günnemann.
Año de publicación	2022
Motivación	Los autores identifican que los pipelines tradicionales de Machine Learning suelen extraer los datos de una base de datos hacia herramientas externas como NumPy, TensorFlow o PyTorch para realizar el entrenamiento. Esta extracción puede resultar costosa cuando se trabaja con grandes volúmenes de información. Por ello, investigan la posibilidad de ejecutar directamente dentro del sistema de base de datos las etapas de preprocesamiento, entrenamiento y validación, aprovechando además procesamiento paralelo en CPU y GPU.
Problema que resuelve	Busca reducir el costo de transferencia y procesamiento requerido para entrenar modelos de Machine Learning sobre grandes conjuntos de datos. Además, estudia cómo paralelizar el entrenamiento mediante múltiples threads, workers y GPUs sin que los mecanismos de sincronización reduzcan excesivamente el rendimiento. El artículo utiliza explícitamente Regresión Lineal como uno de sus modelos de evaluación.
Propuesta de solución	Los autores extienden el sistema de base de datos Umbra para expresar un pipeline completo de Machine Learning mediante SQL. Incorporan diferenciación automática y un operador de Gradient Descent capaz de generar código LLVM y ejecutar el entrenamiento sobre GPU. Para Regresión Lineal, los datos se procesan por mini-batches y se distribuyen entre threads/workers. Se comparan estrategias con sincronización global, actualizaciones sin bloqueo y modelos locales.
Algoritmos usados / pseudocódigo	Se utiliza Regresión Lineal, Mini-batch Gradient Descent, diferenciación automática y técnicas paralelas de reducción/agregación de gradientes. El artículo presenta distintas estrategias de sincronización: workers sincronizados, workers con actualizaciones globales y workers con modelos locales. Cada worker procesa una parte de los datos, calcula su gradiente y

	posteriormente los resultados son agregados para actualizar los pesos del modelo.
Servicios, frameworks, bibliotecas, lenguaje, patrones	Umbral como DBMS experimental; SQL para representar el pipeline; LLVM para generación de código; CUDA para ejecución en GPU y cuBLAS para operaciones vectoriales/matriciales. Se utilizan CPU threads, GPU threads, múltiples workers, particionamiento de datos, mini-batches, reducción de gradientes y modelos globales/locales.
Opinión crítica del estudiante	Considero que el trabajo demuestra de forma clara el beneficio de paralelizar el entrenamiento de Regresión Lineal, especialmente cuando se procesan grandes cantidades de datos. El uso de varios workers permite incrementar el throughput y reducir el tiempo de ejecución; sin embargo, el artículo también evidencia que la concurrencia no garantiza automáticamente un mayor rendimiento. Cuando se utilizan locks o sincronización bloqueante, algunos workers permanecen inactivos esperando a otros, reduciendo el speedup obtenido. Esto muestra que el diseño del mecanismo de sincronización es tan importante como el número de workers utilizados. Esta observación es especialmente relevante para nuestra implementación en Go, donde será necesario controlar correctamente el uso de goroutines, sync.Mutex y sync.WaitGroup.
Link de acceso al paper	https://link.springer.com/article/10.1007/s10619-022-07417-7

Paper 3 (Integrante 3 - Jhamil Brijan Peña Cardenas)

Campo	Contenido
Título del paper	[Completar]
Autores	[Completar]
Año de publicación	[Completar]
Motivación	[Completar]
Problema que resuelve	[Completar]
Propuesta de solución	[Completar]
Algoritmos usados / pseudocódigo	[Completar]

Servicios, frameworks, bibliotecas, lenguaje, patrones	[Completar]
Opinión crítica del estudiante	[Completar]
Link de acceso al paper	[Completar]

Elección Final:

4. Análisis del caso de uso

4.1 Elección y sustentación

El caso de uso seleccionado consiste en el análisis de costos médicos y días de estancia hospitalaria para apoyar la optimización de la asignación de camas y presupuestos en centros de salud, en línea con el ODS 3 (Salud y bienestar). Se eligió este caso por tres razones principales:

- Relevancia social: la gestión eficiente de recursos hospitalarios impacta directamente en la calidad y oportunidad de la atención médica, especialmente en sistemas de salud con recursos limitados.
- Naturaleza tabular y numérica de los datos: el análisis se basa en variables cuantitativas (días de estancia, montos de pago, conteo de diagnósticos), lo que evita el procesamiento de imágenes médicas o texto clínico sensible, reduciendo la complejidad y los riesgos de privacidad.
- Escalabilidad del problema: los sistemas de salud pública generan volúmenes de reclamos del orden de millones de registros anuales, lo que justifica naturalmente el uso de concurrencia para procesar el dataset en tiempos razonables.

El modelo elegido, Regresión Lineal, responde directamente a este caso: permite predecir el costo total de atención (Total Costs) en función de los días de estancia (Length of Stay) y variables categóricas asociadas al diagnóstico (APR DRG) y al tipo de pago (Payment Typology), apoyando directamente la planificación presupuestaria y la estimación anticipada del gasto por caso, insumo clave para la asignación de camas y recursos.

4.2 Selección del dataset

Se evaluó inicialmente el dataset público "Medicare Inpatient Prospective Payment System - Provider Summary" (CMS), que contiene variables agregadas por hospital y DRG (total_discharges, average_covered_charges, average_total_payments). Sin embargo, este dataset solo contiene aproximadamente 163,000 registros (cerca de 3,000 hospitales por ~100

categorías de procedimiento), por lo que no cumple el requisito de más de un millón de registros exigido por el curso.

Se evaluó también el CMS Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), que sí supera holgadamente el millón de registros, pero corresponde a reclamos sintéticos de 2008-2010, fuera de la ventana de vigencia de 5 años que el grupo decidió aplicar de forma consistente tanto a los papers académicos como al dataset.

Por ello, el grupo selecciona como dataset definitivo el "Hospital Inpatient Discharges (SPARCS De-Identified)" del año 2023, publicado por el New York State Department of Health a través del portal público Health Data NY (health.data.ny.gov). El SPARCS (Statewide Planning and Research Cooperative System) es un sistema de reporte obligatorio que recopila el detalle de cada alta hospitalaria en el estado de Nueva York, y su versión identificada es de acceso público y gratuito.

El dataset SPARCS 2023 contiene más de 2.1 millones de registros a nivel de alta hospitalaria individual, con variables directamente alineadas al caso de uso: Length of Stay (días de estancia), Total Charges y Total Costs (cargos y costos totales), APR DRG Description y CCS Diagnosis/Procedure Description (diagnóstico y procedimiento, usados únicamente como categorías/conteos, sin texto clínico libre), Payment Typology (tipo de pago/asegurador) y Facility Id (identificador del hospital). Al ser datos reales de un solo año calendario reciente, el dataset cumple simultáneamente el requisito de más de un millón de registros y la ventana de vigencia de 5 años aplicada al trabajo.

Como respaldo académico de la elección, un estudio publicado en BMC Health Services Research (2024) utilizó una fuente equivalente del mismo sistema SPARCS (2.3 millones de registros de altas hospitalarias del estado de Nueva York) para predecir el length of stay mediante modelos de machine learning, validando la pertinencia de esta fuente para el caso de uso planteado.

5. Procedimiento de limpieza del dataset

Dado que el curso exige el uso de concurrencia en Go desde el planteamiento del algoritmo, el procedimiento de limpieza y consolidación del dataset se implementará como un pipeline concurrente en Go (no como un script secuencial de preprocesamiento), reutilizando la misma arquitectura que luego se comparará contra su versión secuencial en el Entregable 2 (PC2).

5.1 Pasos de depuración a aplicar

- Carga: leer el archivo CSV de altas hospitalarias SPARCS 2023 (>2.1 millones de registros) descargado desde Health Data NY.
- Selección de variables: conservar únicamente las columnas relevantes para el modelo (Length of Stay, Total Charges, Total Costs, APR DRG Description, Payment Typology, Facility Id), descartando campos no utilizados para reducir el tamaño en memoria.

- Tratamiento de valores nulos: los registros con Length of Stay, Total Charges o Total Costs vacíos o no numéricos se descartan; los campos categóricos vacíos (por ejemplo Payment Typology) se etiquetan como "No especificado".
- Tratamiento de outliers: se identificarán valores atípicos en días de estancia y en costos/cargos totales (por ejemplo mediante rango intercuartílico), dado que este dataset incluye casos extremos reales (estancias muy prolongadas, procedimientos de alto costo), documentando el criterio de exclusión o tratamiento usado.
- Normalización/escalado: las variables numéricas predictoras (Length of Stay, Total Charges) se escalarán (estandarización z-score o min-max) para favorecer la convergencia del entrenamiento por descenso de gradiente de la Regresión Lineal, especialmente al paralelizar el cálculo de sumas parciales entre goroutines.
- Codificación de categóricas: variables como APR DRG Description, Payment Typology y Facility Id se codificarán (one-hot o conteo por categoría) para su uso como variables predictoras o de agrupamiento.
- Partición: separación en conjuntos de entrenamiento y prueba (por ejemplo 80/20) para la validación posterior del modelo de Regresión Lineal.

5.2 Arquitectura concurrente en Go (Worker Pool + Pipeline)

El pipeline de limpieza se estructura en tres etapas comunicadas por canales, siguiendo el patrón Worker Pool para la etapa de lectura/depuración y un consumidor único para la escritura, evitando condiciones de carrera:

- Etapa 1 - Worker Pool de lectura y depuración: se lanzan N goroutines (una por CPU lógico disponible, vía runtime.NumCPU()), y el archivo CSV de SPARCS 2023 se reparte en bloques de filas entre ellas (por ejemplo, dividiendo el archivo en N particiones o repartiendo filas de forma round-robin desde un lector central). Cada worker parsea su bloque con encoding/csv, aplica los filtros de nulos/outliers y transforma sus filas de forma independiente, sin compartir estado con las demás goroutines.
- Comunicación - Canal de filas limpias: cada worker envía las filas depuradas a un chan[]string compartido (con buffer, para desacoplar velocidad de lectura y escritura).
- Sincronización - sync.WaitGroup: coordina el cierre del canal una vez que todas las goroutines de lectura terminan (wg.Wait() antes de close(chanFilas)), evitando el cierre prematuro del canal.
- Sincronización - sync.Mutex: protege contadores compartidos en memoria (registros leídos, descartados por nulos, descartados por outliers) que cada worker incrementa al finalizar.
- Etapa 2 - Consumidor único de escritura: una sola goroutine lee del canal y escribe el CSV consolidado final, garantizando que no haya escrituras concurrentes al mismo archivo.
- Medición: se registrará el tiempo total de ejecución de este pipeline (time.Since) tanto en su versión concurrente como en una versión secuencial equivalente (un solo hilo, un archivo tras otro), como primer insumo de speedup para el Entregable 2.

Este procedimiento se documentará con evidencia reproducible (código Go versionado en GitHub, capturas de ejecución) y se reportará el número de registros antes y después de cada etapa de depuración, además del tiempo de ejecución obtenido.

6. Trabajo en GitHub

El proyecto se aloja en un repositorio público de GitHub siguiendo el flujo Git Flow (ramas main/master, develop, feature/*, release/*, hotfix/*). Cada integrante realiza commits desde su propia cuenta para evidenciar su participación individual, conforme a lo exigido por la rúbrica.

Link del repositorio: <https://github.com/YuhiTTo/Grupo5-ProgramacionConcurrente.git>

7. Referencias bibliográficas (formato APA)

Nguyen, M., Kirst, C., & Mücke, N. (2024). Distributed SGD in overparametrized linear regression. *Analysis and Applications*, 22(3), 425–466. <https://www.worldscientific.com/doi/10.1142/S021953052350032X>

Schüle, M. E., Lang, H., Springer, M., Kemper, A., Neumann, T., & Günemann, S. (2022). Recursive SQL and GPU-support for in-database machine learning. *Distributed and Parallel Databases*, 40(2–3), 205–259. <https://link.springer.com/article/10.1007/s10619-022-07417-7>